Vision-Based Distracted Driving Detection: A Comparative Analysis of Deep Learning Object Detection Architectures

Kien Pham
Department of Computer Engineering and Computer Science
California State University, Long Beach
Long Beach, USA
kien.pham02@student.csulb.edu

Amir Ghasemkhani, Ph.D.
Department of Computer Engineering and Computer Science
California State University, Long Beach
Long Beach, USA
amir.ghasemkhani@csulb.edu

***Abstract*—Distracted driving represents a critical road-safety challenge in the United States, with the National Highway Traffic Safety Administration (NHTSA) reporting 3,275 deaths in 2023 from crashes involving distracted drivers. This work addresses this problem through vision-based distraction detection by conducting a comprehensive empirical evaluation of three prominent deep learning object detection architectures on the Roboflow Mobile Detection dataset. We compare Faster R-CNN, a two-stage detector, with two single-stage detectors (Single Shot MultiBox Detector and YOLOv11) to systematically analyze the fundamental trade-offs between detection accuracy and real-time inference performance. Our results demonstrate that YOLOv11 achieves the best overall balance, delivering the highest mean average precision (mAP = 0.604) while maintaining an exceptional inference speed (86.1 FPS), making it the most suitable architecture for real-time driver monitoring systems. This comparative study provides empirical guidance for practitioners deploying object detection systems in safety-critical applications with stringent latency requirements.**

***Index Terms*—object detection, distracted driving, Faster R-CNN, Single Shot MultiBox Detector, YOLO, in-cabin monitoring, real-time detection**

I. Introduction

Distracted driving remains one of the leading causes of motor vehicle crashes and fatalities in the United States. According to recent statistics from the National Highway Traffic Safety Administration (NHTSA), approximately 3,275 people died from distracted crashes in 2023 alone [1]. Beyond obvious hazards such as mobile phone usage, many distraction behaviors occur only fractions of a seconds which can be difficult for human observers to detect. This includes behaviors such as eating, drinking, smoking, and hand gestures. Detecting such behaviors in real-world conditions is particularly challenging due to in-cabin environmental factors, including variable lighting, motion blur, partial occlusion from hands and steering wheels, and varying camera viewpoints. Hence, reliably identifying brief and subtle distraction behaviors remains a challenging task, highlighting the need for automated approaches that can operate consistently across diverse in-cabin conditions.

Automated driver distraction detection systems offer significant potential to improve road safety by enabling timely alerts or automatic interventions. However, the effectiveness of such systems depends critically on two key factors: (1) detection accuracy which is the ability to reliably identify distraction behaviors across diverse conditions, and (2) real-time performance which is the ability to process video frames at sufficient speed to capture short-duration events and trigger timely responses. This dual requirement creates an inherent design tension which requires a tradeoff between inference accuracy and speed. Traditional two-stage detectors like Faster R-CNN prioritize accuracy through region proposal mechanisms but incur substantial computational overhead. Conversely, single-stage detectors like Single Shot MultiBox Detector (SSD) and YOLO sacrifice some accuracy for speed. Choosing an appropriate architecture for a deployed system requires understanding these trade-offs quantitatively.

Object detection has evolved considerably over the past decade, progressing from traditional hand-crafted features to deep learning-based approaches. Early work by Krizhevsky et al. [2] demonstrated the effectiveness of convolutional neural networks (CNNs) for image classification which in turn established the foundation for modern object detection methods. The two-stage detection paradigm was pioneered by Girshick et al. with R-CNN [3], which leveraged selective search to propose candidate regions and classified each using a CNN. This approach was later refined into Fast R-CNN [4] and Faster R-CNN [5], which introduced the Region Proposal Network (RPN) to generate proposals end-to-end, significantly improving both accuracy and speed.

On the other hand, single-stage detectors emerged as an alternative approach, aiming to eliminate the region proposal step entirely. Liu et al. introduced the Single Shot MultiBox Detector (SSD) [6], which makes predictions directly on feature maps at multiple scales. Around the same time, Redmon et al. [7] introduced YOLO (You Only Look Once), framing object detection as a single regression problem, achieving real-time performance on consumer hardware. YOLO has since evolved through multiple versions, with improvements in accuracy and speed through architectural innovations [8], [9]. Recent advances have further refined these architectures. Attention mechanisms have been integrated into detection networks [10], enabling models to focus on the most informative regions. Additionally, multi-scale feature fusion strategies have been shown to improve detection of objects at varying scales [11].

In the context of driver monitoring and distraction detection specifically, several prior works have explored computer vision

approaches. Studies by Jain et al. [12] and Gadde et al. [13] have investigated gaze estimation and head pose detection as indicators of driver attention. However, fewer works have conducted systematic benchmarking of modern object detection architectures specifically for in-cabin distraction classification on recent datasets. To further enrich the literature, we conduct a rigorous empirical comparison of three widely-used object detection architectures (Faster R-CNN, SSD, and YOLOv11) which are specifically tailored to the distracted driving detection problem. We provide quantitative analysis of both accuracy (mAP at multiple IoU thresholds) and inference speed performance (FPS and per-image latency), offering practical guidance for practitioners selecting architectures for real-time safety-critical applications.

The remainder of this paper is organized as follows: Section II describes the three detection architectures in detail, providing both architectural insights and mathematical formulations. Section **??** details our experimental setup, dataset, and evaluation metrics. Section III presents our comparative results and analysis. Finally, Section IV summarizes findings and recommendations.

II. Deep Learning Object Detection Architectures

This section provides detailed descriptions of the three object detection architectures evaluated in this work. We begin with the mathematical formulation of the object detection problem, then describe each architecture in turn.

A. Problem Formulation

We consider the object detection problem as learning a mapping from an input image to a set of object instances. Given an image $\mathbf{X} \in \mathbb{R}^{m \times n \times 3}$ (with spatial dimensions $m \times n$ and three RGB channels), the ground-truth annotation is a variable-size set

$$y = \{(\mathbf{b}_j, c_j)\}_{j=1}^{N}, \tag{1}$$

where N is the number of objects in $\mathbf{X}$, $c_j \in \{1, \ldots, C\}$ is the class label for the j-th object among C classes, and $\mathbf{b}_j$ denotes its bounding box. We represent each bounding box as

$$\mathbf{b}_j = (x_j, y_j, w_j, h_j), \tag{2}$$

where (x_j, y_j) are the center coordinates and (w_j, h_j) are the width and height (typically expressed in pixels or normalized to $[0, 1]$).

A detector parameterized by weights θ predicts a set of candidate detections,

$$\hat{y} = f(\mathbf{X}; \theta), \tag{3}$$

which is subsequently filtered (e.g., by score thresholding and non-maximum suppression) to yield the final set of detections. While the formulation of the detection task is shared across modern approaches, different object detection architectures implement the mapping $f(\mathbf{X}; \theta)$ using distinct design strategies. These differences primarily concern how features are extracted, how bounding boxes and class scores are predicted, and whether detection is performed in one or multiple stages. In what's following, we describe three representative detection architectures where both two-stage and one-stage paradigms and their key design principles are discussed.

B. Faster R-CNN: Two-Stage Detection

Faster R-CNN [5] represents the two-stage detection strategy in which object detection is decomposed into two sequential stages: (1) region proposal generation, and (2) region classification and refinement.

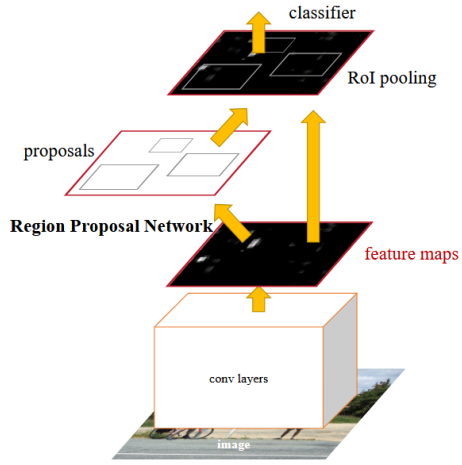


Fig. 1. Faster-RCNN [5]

1) Architecture Overview: The network comprises three main components. First, a shared convolutional backbone (typically VGG-16 or ResNet) extracts a convolutional feature map from the input image. In our implementation, we use VGG-16 as the backbone. Second, a Region Proposal Network (RPN) slides a small window over the feature map and, at each position, predicts multiple candidate object regions (bounding boxes) using a set of predefined anchors with different scales and aspect ratios. Third, a Faster R-CNN head then classifies each proposed region into object categories and refines the bounding box coordinates through regression. Figure 1 illustrates the architecture flow of faster R-CNN method.

2) Region Proposal Network: The RPN operates on the convolutional feature map, generating proposals for the second stage. At each sliding-window position, the RPN predicts k anchor boxes with varying scales and aspect ratios. For each anchor, the RPN predicts:

- **Objectness scores**: A 2-way softmax output indicating probability of foreground vs. background.
- **Bounding box offsets**: 4 regression values (d_x, d_y, d_w, d_h) that refine the anchor coordinates.

With $k = 9$ anchors per location (3 scales $\times$ 3 aspect ratios, the default configuration), a feature map with spatial dimensions $H \times W$ produces $H \times W \times 9 \times 4$ regression outputs and $H \times W \times 9 \times 2$ classification outputs.

Non-Maximum Suppression (NMS) is applied post-RPN to remove redundant proposals, yielding a refined set of region proposals for the second stage.

3) Fast R-CNN Head and Classification: The Fast R-CNN head processes each region proposal by extracting a fixed-size feature vector (typically via RoI Pooling) from the feature map. This vector is passed through fully-connected layers to jointly predict:

- **Class probabilities**: A softmax distribution over object classes (including background).
- **Bounding box refinement**: Per-class bounding box regression offsets.

Finally, NMS is applied to the final predictions to produce

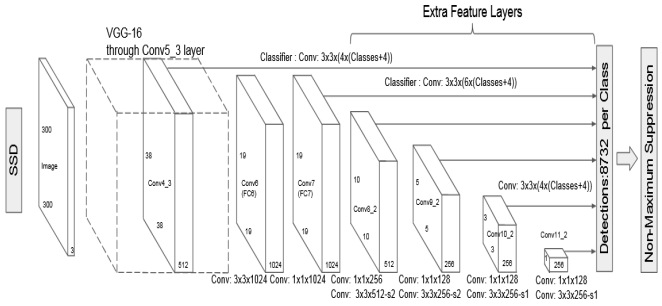


Fig. 2. Single Shot MultiBox Detector (SSD) [6]

the set of detected objects.

4) Comparison with Other Approaches: Faster R-CNN's two-stage design offers distinct advantages and trade-offs. By explicitly generating region proposals, the model focuses computational resources on candidate regions, which can improve accuracy in detecting small or occluded objects. This is specifically relevant for in-cabin distraction detection where hands or phones may be partially visible. However, the sequential nature of the two stages and the computational cost of processing each proposal through the second stage result in higher latency compared to single-stage approaches. This makes Faster R-CNN less suitable for real-time applications but valuable as a high-accuracy baseline.

C. Single Shot MultiBox Detector (SSD)

SSD [6] pioneered the single-stage detection strategy, eliminating the region proposal step and predicting object locations and classes directly from convolutional feature maps at multiple scales.

1) Architecture Overview: SSD applies detection heads directly to convolutional feature maps extracted at multiple depths of the network. Starting from a base convolutional backbone (e.g., VGG-16 up to the $conv5_3$ layer), SSD appends a sequence of additional convolutional layers that progressively reduce spatial resolution while increasing receptive field. Together, these layers form a multi-scale hierarchy of feature maps. Unlike two-stage detectors such as the original Faster R-CNN, which perform detection from a single high-level feature map, SSD predicts object locations and class scores independently from each feature map in this hierarchy. Detection heads are applied to each feature map, enabling the model to capture objects at multiple scales. This results in higher-resolution feature maps to be more sensitive to small objects, while lower-resolution maps capture larger objects. Figure 2 illustrates this multi-scale detection pipeline.

2) Multi-Scale Detection Head: For each feature map with spatial dimensions $H \times W$, using k predefined default boxes (anchors) per location, and making predictions over C classes including background, the detection head outputs:

$$H \times W \times k \times (C + 4) \tag{4}$$

values, where the first C values are class confidence scores and the remaining 4 values encode bounding box coordinates. This prediction is computed for each of the feature maps, with smaller feature maps capturing larger objects and larger feature maps capturing smaller objects in the multi-scale hierarchy. Predictions from all feature maps are then aggregated and filtered using non-maximum suppression (NMS) to produce the final set of detections.

3) Comparison with Other Approaches: SSD's single-stage architecture eliminates the overhead of region proposal generation, resulting in faster inference than Faster R-CNN. However, because the detection head makes predictions at every spatial location, SSD can struggle with small objects and may produce more false positives in crowded scenes compared to region-proposal based methods. For distraction detection, this trade-off is acceptable in many scenarios where speed is often prioritized over marginal accuracy gains, and the distraction behaviors (e.g., drinking, eating, phone usage) typically occupy limited and predictable regions of the image (e.g., near the driver's hands and face), reducing the impact of SSD's weaknesses on small or crowded objects.

D. YOLOv11: Modern Single-Stage Detection

YOLOv11 is a recent evolution of the YOLO family, incorporating modern architectural components such as efficient feature fusion and attention-style modules for improved representation quality in real-time detection settings.

1) Architecture Overview: YOLOv11 follows a three-component decomposition commonly used to describe YOLO-style detectors: a backbone for feature extraction, a neck for multi-scale feature fusion, and a detection head for multi-scale predictions. Figure 3 shows architectural flow of YOLO11 method.

a) Backbone: Feature Extraction: The backbone progressively downsamples the input image through convolutional stages, producing hierarchical feature maps at multiple scales. Earlier layers encode low-level visual patterns (e.g., edges and textures), while deeper layers encode higher-level semantic structure. Downsampling reduces spatial resolution while increasing channel depth, improving computational efficiency while enabling larger receptive fields. YOLOv11 uses `C3k2` blocks which are CSP-style (Cross Stage Partial) convolutional blocks, as its primary building module throughout the network. These blocks split feature channels into parallel paths, allowing only a subset to pass through deeper transformations before being recombined leading to improved efficiency and

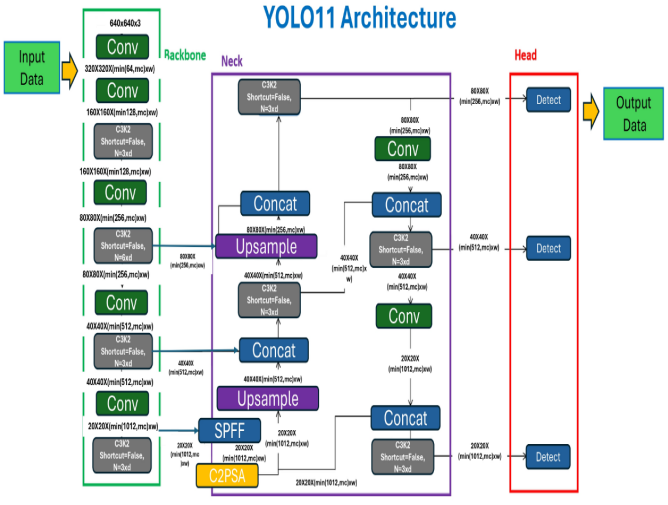


Fig. 3. YOLO11 [16]

gradient flow. These CSP-style blocks are designed to balance representational capacity and computational cost via structured feature mixing and repeated bottleneck-style sub-blocks.

Following the main downsampling stages, YOLOv11 employs an `SPPF` (Spatial Pyramid Pooling–Fast) module to enrich contextual information using repeated pooling at a fixed kernel size. Unlike the classic SPP module (which uses parallel pooling with multiple kernel sizes), SPPF computes a similar multi-receptive-field effect using sequential pooling, then concatenation and point-wise mixing. In the implemented forward pass, this can be expressed as:

$$y_0 = \mathrm{Conv}_1(x), \tag{5}$$

$$y_i = \mathrm{MaxPool}_k(y_{i-1}), \quad i = 1, \ldots, n, \tag{6}$$

$$y = \mathrm{Conv}_2\Big(\mathrm{Concat}(y_0, y_1, \ldots, y_n)\Big), \tag{7}$$

$$\mathrm{SPPF}(x) = \begin{cases} y + x & \text{if a shortcut is enabled} \\ & \text{and shapes match,} \\ y & \text{otherwise.} \end{cases} \tag{8}$$

In addition, YOLOv11 can incorporate attention-style blocks such as `C2PSA`, which apply parallel spatial attention mechanisms to which apply spatial attention mechanisms to adaptively emphasize informative regions of the feature map while suppressing less relevant background responses.

b) Neck: Feature Fusion Across Scales: To support robust detection over a range of object sizes, YOLOv11 fuses features from different backbone depths using upsampling and concatenation. This combines semantically rich deep features with higher-resolution shallow features:

$$f_{\text{fused}} = \mathrm{Concat}\big(\mathrm{Upsample}(f_{\text{deep}}),\ f_{\text{shallow}}\big), \tag{9}$$

where concatenation is performed along the channel dimension. Subsequent `C3k2` refinement blocks further process these fused representations. This multi-scale fusion is essential for simultaneously detecting small objects and larger context cues.

c) Detection Head: Multi-Scale Predictions: The detection head operates on multiple feature-map scales (typically three: $P3$, $P4$, and $P5$), with each scale focusing on different object-size regimes. For each scale $s \in \{P3, P4, P5\}$, the head produces raw regression outputs (for bounding-box localization) and class logits:

$$\{r_s,\ \ell_s\} = \mathrm{Head}(f_s), \qquad s \in \{P3, P4, P5\}, \tag{10}$$

where r_s denotes the raw box-regression outputs and ℓ_s denotes the class logits at scale s.

These outputs are then converted into final detections by decoding the box regression and applying a sigmoid to obtain per-class probabilities:

$$\hat{B}_s = \mathrm{DecodeBoxes}(r_s;\ a_s, \mathrm{stride}_s), \tag{11}$$

$$\hat{P}_s = \sigma(\ell_s), \qquad s \in \{P3, P4, P5\} \tag{12}$$

where a_s and stride_s are the anchor/grid parameters associated with scale s, and $\sigma(\cdot)$ is the element-wise sigmoid.

2) Comparison with Other Approaches: YOLOv11 retains the speed advantages typical of single-stage detectors while leveraging modern feature fusion and attention-style modules to improve representation quality. In general, single-stage detectors (e.g., YOLO/SSD families) are typically faster than two-stage detectors (e.g., Faster R-CNN family), while the accuracy/speed trade-off depends on the dataset, model scale, training recipe, and deployment constraints. Compared to earlier single-stage designs such as SSD, YOLOv11 benefits from a more modern backbone and multi-scale fusion strategy optimized for real-time performance.

III. Results and Analysis

A. Dataset

This work uses the Mobile Detection Computer Vision Model dataset hosted on Roboflow, which was collected specifically for driver distraction detection in in-cabin settings. The dataset contains four distraction-related classes: *drinking*, *eating*, *smoking*, and *mobile* (phone usage) [15]. These classes represent common secondary activities that can impair driver attention and are representative of real-world in-vehicle scenarios.

The dataset exhibits a noticeable class imbalance across the training, validation, and test splits. In the training set, the number of samples per class are 978 (*smoking*), 1,817 (*drinking*), 1450 *eating*, and 1150 *mobile* usage. Similar imbalance patterns are observed in the validation and test splits, where *drinking* remains the most frequent class and *smoking* the least frequent. Such imbalance reflects natural differences in the occurrence of these behaviors but may bias the learning process toward majority classes and degrade performance on underrepresented activities. To mitigate this issue during training, we apply class-level oversampling to the training split, equalizing the number of samples across all classes to 1,817. The validation and test distributions are left unchanged to preserve a realistic evaluation setting. This approach encourages the model to learn more discriminative and robust features for minority distraction categories.

B. Experimental Setup

To ensure fair comparison across the three architectures, we maintain a consistent experimental protocol:

- **Data Split**: We use the same train-validation-test split across all models.
- **Preprocessing**: All images are resized to a consistent resolution, and pixel values are normalized using standard ImageNet statistics.
- **Augmentation**: We apply a consistent augmentation pipeline including random cropping (to 300×300), horizontal flipping (probability 0.5), and brightness/contrast adjustment (probability 0.5). These augmentations are chosen to simulate realistic variations in cabin imagery.
- **Training**: Each model is trained with the same number of epochs and learning rate schedule.

C. Evaluation Metrics

We evaluate performance using two complementary metrics:

- **Accuracy**: We report mean Average Precision (mAP) computed at IoU thresholds from 0.50 to 0.95 in increments of 0.05 (the standard COCO evaluation protocol [14]). This metric captures detection quality across varying levels of localization precision.
- **Speed**: We report frames per second (FPS) and per-image inference time (in seconds), both critical for real-time deployment.

TABLE I
COMPARATIVE PERFORMANCE OF OBJECT DETECTION ARCHITECTURES

Model	Params (M)	mAP	FPS	Runtime (s)	Inference Time (s)
SSD	35	0.427	29.4	2.31	0.0340
Faster R-CNN	43	0.443	5.24	13.0	0.1910
YOLOv11	20	0.604	86.1	1.39	0.0116

D. Quantitative Results

Table I summarizes the performance of the three architectures on the distraction detection task.

1) Single Shot Detector (SSD) Analysis: SSD, with 35 million parameters, achieves a balance between computational efficiency and detection accuracy. It achieved an mAP of 0.427 with inference at 29.4 FPS (0.034 s per image). This performance is sufficient for real-time video processing at standard frame rates (24–30 FPS), enabling smooth in-cabin monitoring.

The speed of SSD is a significant advantage for deployment. Compared to Faster R-CNN, SSD runs over 5× faster (29.4 FPS vs. 5.24 FPS), which directly translates to more consistent frame coverage and reduced latency in triggering safety alerts.

However, SSD's accuracy (mAP = 0.427) is notably lower than both Faster R-CNN and YOLOv11. This can lead to higher false negative rates, potentially missing subtle distraction behaviors in challenging in-cabin conditions (occlusion, motion blur, poor lighting). For safety-critical applications where sensitivity to distraction events is paramount, this accuracy trade-off may be problematic.

2) Faster R-CNN Analysis: Faster R-CNN, with 43 million parameters, achieved an mAP of 0.443 which is the highest accuracy among two-stage detectors and only marginally better than SSD (+3.8 percentage points). This modest improvement comes at substantial computational cost: Faster R-CNN operates at only 5.24 FPS (0.191 s per image), roughly 17× slower than YOLOv11.

For real-time in-cabin monitoring applications, 5.24 FPS presents a significant limitation. At this frame rate, the system may miss short-duration distraction events (e.g., a brief glance at a phone lasting 0.5 seconds could be missed if sampled infrequently). Furthermore, the latency of 0.191 s per image introduces noticeable delay between when a behavior occurs and when an alert is triggered—potentially too slow for preventive intervention in some scenarios.

However, Faster R-CNN remains valuable as a high-accuracy reference model for validation and for scenarios where absolute accuracy is prioritized over latency (e.g., post-hoc video analysis or offline assessment of driving behavior).

3) YOLOv11 Analysis: YOLOv11, with only 20 million parameters (roughly 47% and 53% fewer parameters than SSD and Faster R-CNN, respectively), achieves the best overall performance. It attained an mAP of 0.604 which is a substantial improvement over SSD and Faster R-CNN. Simultaneously, YOLOv11 is the fastest model, operating at 86.1 FPS with a per-image inference time of just 0.0116 seconds.

The superior accuracy of YOLOv11 reflects the effectiveness of modern architectural innovations. The multi-scale feature fusion in the neck enables the model to detect distraction-related objects across a range of sizes—from small phone-in-hand scenarios to larger body postures. Attention mechanisms in the backbone help the model focus on task-relevant features, suppressing background clutter common in in-cabin imagery.

The speed advantage of YOLOv11 is equally compelling. At 86.1 FPS, the system can process typical 30-FPS video streams with 2.87× oversampling, providing robust coverage of all frames and capacity for processing video from multiple cabin cameras simultaneously. The 0.0116 s latency is imperceptible to human observers and enables rapid response to detected distraction events.

IV. CONCLUSION

This study provides an empirical comparison of three object detection architectures for distracted driving detection. Faster R-CNN achieves higher accuracy at the cost of high inference latency, while SSD offers moderate performance with improved speed. YOLOv11 demonstrates the most favorable balance between detection accuracy and inference efficiency. Future work will focus on improving performance through domain adaptation, incorporating temporal information, and integrating complementary sensing modalities to enhance robustness in practical deployments.

REFERENCES

[1] National Highway Traffic Safety Administration, "Traffic Safety Facts: Distracted Driving 2023," 2024. [Online]. Available: https://www.nhtsa.gov

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[3] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 580–587.

[4] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1440–1448.

[5] S. Ren, K. He, X. Sun, and J. Zhang, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[6] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. E. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 21–37.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.

[8] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7263–7271.

[9] A. Bochkovskiy, C.-Y. Wang, and H.-M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[10] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11534–11542.

[11] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2117–2125.

[12] A. Jain, A. Tran, M. Tanveer, T. Hasan, R. St-Charles, A. Fung, M. H. Merzban, M. Yuen, and A. Videoprocessing, "Supervised domain adaptation for gaze estimation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2016, pp. 3725–3733.

[13] R. Gadde, V. Jampani, and P. V. Gehler, "Semantic video segmentation by gated recurrent flow propagation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4973–4982.

[14] T.-Y. Lin, M. Maire, S. Belongie, J. Hao, P. Perona, C. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.

[15] asu, "Mobile Detection Dataset," *Roboflow Universe*, Roboflow, Jun. 2023. [Online]. Available: https://universe.roboflow.com/asu-b6mtv/mobile-detection-l2iov. [Accessed: 2025].

[16] S. Cepeda, O. Esteban-Sinovas, R. Romero, V. Singh, P. Shetty, A. Moiyadi, I. Zemmoura, G. R. Giammalva, M. Del Bene, A. Barbotti, F. DiMeco, T. R. West, B. V. Nahed, I. Arrese, R. Hornero, and R. Sarabia, "Real-Time Brain Tumor Detection in Intraoperative Ultrasound Using YOLO11: From Model Training to Deployment in the Operating Room," *arXiv* preprint arXiv:2501.15994 [eess.IV], Jan. 2025. doi:10.48550/arXiv.2501.15994. https://arxiv.org/abs/2501.15994